vdot.vv 协同仿真和性能分析

1. 测试环境

项目	配置
处理器	香山昆明湖 V2，MinimalConfig，单核
向量宽度	VLEN=128
点积实现	复用 VFEX0/VIMAC，按 vl 计算 INT8 点积，结果为 32 位
NEMU	`riscv64-xs-vdot-ref_defconfig`，开启向量寄存器及 CSR 比较
功能应用	`apps/vdot-csr`，`BASIC=1`，共 6 项
性能应用	`apps/vdot-bench`，128 组输入，每次遍历 2 遍，各测 3 轮

测试使用支持 `vdot.vv` 的香山仿真器和 NEMU，通过差分检查指令执行结果，并比较 RVV 组合指令与 `vdot.vv` 完成同一批点积运算的周期数。

2. NEMU 指令扩展

2.1 修改内容

补丁 nemu-vdot.patch 基于提交 `2ec110b618bb8f08cf0b54ca3d9cbb0f210f895d`，涉及以下 6 个文件：

文件	作用
`src/isa/riscv64/instr/decode.c`	在 Custom-0 空间识别点积编码，复用 OP_V 提取操作数
`src/isa/riscv64/include/isa-all-instr.h`	登记 vdot 执行入口
`src/isa/riscv64/instr/rvv/vcompute.h`	根据 vl 和 LMUL 读取源组、求点积并更新目标
`src/cpu/difftest/ref.c`	接收 RTL 实际使用的启动镜像
`configs/riscv64-xs-vdot-ref_defconfig`	保存与当前香山匹配的参考模型配置
`.gitignore`	允许跟踪该专用配置文件

2.2 指令行为

执行函数先检查向量状态、`vill`、`vm=1`、`SEW=8`、合法 LMUL、`vstart=0` 和源寄存器组对齐，再根据 `vl` 执行运算。不满足指令要求时，触发非法指令异常。

当 `vl>0` 时，使用 `get_vreg` 读取两个源组中的前 `vl` 个 INT8 元素，计算点积后写入目标。目标为单个向量寄存器；先读完源数据再写回，允许源目标重叠。

当 `vl=0` 时保留目标寄存器原值；当 `vl>0` 时，低 32 位保存点积，高 96 位清零，不受 `vta/vma` 影响。执行完成后清零 `vstart`，保持 `vl/vtype/vxrm/vxsat`，将向量上下文标为脏。

2.3 启动与配置

初始化时，通过 `difftest_load_flash_v2` 将香山的启动镜像传给 NEMU，使两者执行相同的启动代码。

NEMU 配置启用 `CONFIG_NMIE_INIT`、关闭 `CONFIG_MDT_INIT`，使复位状态与香山一致，并开启向量寄存器和向量 CSR 比较。

```
cd /home/xpg/cie/xs-env
source env.sh
make -C "$NEMU_HOME" riscv64-xs-vdot-ref_defconfig
make -C "$NEMU_HOME" -j16
```

3. 基础协同仿真

3.1 测试内容

使用 `BASIC=1` 运行正数、负数和四组边界值测试，共 6 项。

输入	配置	期望值
1…16 与 1…16	LMUL=1，vl=16	1496
−1…−16 与 1…16	LMUL=1，vl=16	−1496
全部为 127	LMUL=8，vl=128	2064512
全部为 −128	LMUL=8，vl=128	2097152
−128 与 127	LMUL=8，vl=128	−2080768
0 与 −128	LMUL=8，vl=128	0

测试程序检查完整目标寄存器和相关 CSR，点积参考值由 C 标量代码计算。协同仿真逐条比较 NEMU 与 RTL 的架构状态。测试代码设计见 基础测试文档。

3.2 运行命令

```
make -C "$AM_HOME/apps/vdot-csr" ARCH=riscv64-xs BASIC=1 disasm
"$NOOP_HOME/build/emu" \
  -i "$AM_HOME/apps/vdot-csr/build/vdot-basic-riscv64-xs.bin" \
  --diff="$NEMU_HOME/build/riscv64-nemu-interpreter-so" \
  --max-cycles=200000 \
  > "$XS_PROJECT_ROOT/vdot-docs/logs/vdot-basic-difftest.log" 2>&1
```

3.3 结果

6 项全部通过，仿真器正常返回 0：

```
The first instruction of core 0 has commited. Difftest enabled.
PASS: positive INT8 dot, VL=16, result=1496 (0x000005d8)
PASS: negative INT8 dot, VL=16, result=-1496 (0xfffffa28)
PASS: boundary values, LMUL=8, VL=128, 4 checks
PASS: vdot basic 6 checks
HIT GOOD TRAP at pc = 0x80000830
```

日志见 基础差分日志，版本和校验值见 基础差分结果记录。协同仿真范围为表中 6 项，尚未测试零长度、其他 LMUL 和非法指令等情况。

4. 现有 RVV 指令基线

两种实现均计算两个 128 位源向量中全部 16 对 INT8 元素的点积，输出低 32 位结果、高 96 位零。每个输入块都使用相同的加载与保存指令：

```
vl1re8.v v2, (t2)
vl1re8.v v3, (t3)
# 点积计算
vs1r.v v4, (t4)
```

4.1 Baseline

在计算函数开始时设置 `vl=16、SEW=16、LMUL=2、tu,mu`，清零归约初值 v12 和目标 v4/v5。每个块执行：

```
vsext.vf2 v8, v2
vsext.vf2 v10, v3
vmul.vv v8, v8, v10
vwredsum.vs v4, v8, v12
```

两次符号扩展将 16 个 INT8 分别扩展为 INT16，每组占两个寄存器。乘积范围为 −16256…16384，INT16 可以完整保存。加宽求和归约得到 INT32 结果。

循环前清零 v4/v5，每次归约更新低 32 位，使用 `tu` 保持其余位为零。每条加载指令读取一个 128 位源寄存器，循环内保持向量配置不变。

4.2 vdot

在计算函数开始时设置 `vl=16、SEW=8、LMUL=1、tu,mu`，每个块用一条指令替换上述计算序列：

```
.word 0x0221820b    # vdot.vv v4, v2, v3
```

两种实现的输入、输出、加载、保存及标量循环相同。每个块均有 3 条向量访存指令和 5 条标量循环指令，计算部分从 4 条指令减少到 1 条，块循环从 12 条减少到 9 条。

块内计算指令经译码拆分后的微操作数量如下：

计算指令	每块微操作数
两条 vsext.vf2，LMUL=2	4
一条 vmul.vv，LMUL=2	2
一条 vwredsum.vs，LMUL=2	4
基线合计	10
一条 vdot.vv，LMUL=1	1

初值和目标清零位于循环外，计入整个任务的总周期，不计入上表的每块微操作数。

5. 批量程序耗时测量

5.1 工作负载

输入包括正数、负数、四组边界值及 122 组固定种子随机数据，共 128 组，每组含两个 128 位向量。

每轮测量遍历全部输入 2 遍，完成 256 次点积，即 4096 次 INT8 乘法。两份输入和一份输出合计 6 KiB；每次点积加载 32 字节、保存 16 字节，每轮的逻辑访存量为 12 KiB。MinimalConfig 的一级数据缓存为 32 KiB，测量前先预热数据。

两种实现各预热一遍，再交替执行 3 轮正式测量。每次运行结束后，检查 128 个目标寄存器的完整结果。两次预热与六次正式测量合计检查 1024 个向量结果；正式测量执行 1536 次点积，预热执行 256 次。

5.2 计时范围

计时包含向量配置、基线初值和目标清零、循环、数据加载、点积和结果保存。启用向量状态、清零 `vstart`、输入初始化、标量参考计算、输出填充、结果检查及打印均在计时外。

该版本香山允许 `rdcycle` 乱序执行。计时边界加入 `fence`，利用其清空流水线的行为约束计数器读取顺序：

```
fence
rdcycle t0
fence
# 整个批量计算任务
fence
rdcycle t1
fence
```

两种实现均以 `t1-t0` 记录批量计算的总周期，包含区间内的计时边界开销。测量时开启 NEMU 差分检查，加速比根据 RTL 周期数计算。

5.3 数据提取

```
bash nexus-am/apps/vdot-bench/run-stage3.sh
```

脚本编译批量测试程序，运行协同仿真，并调用以下命令提取周期数据：

```
python3 nexus-am/apps/vdot-bench/analyze.py \
  vdot-docs/logs/vdot-bench-difftest.log \
  --out-dir vdot-docs/stage3
```

统计以三轮总周期中位数计算：

```
程序加速比 = Baseline 总周期中位数 / vdot 总周期中位数
周期减少比例 = (Baseline 中位数 − vdot 中位数) / Baseline 中位数
```

6. 性能结果与分析

6.1 测量结果

1024 个完整向量结果检查全部通过，NEMU 与 RTL 差分未报错。程序在 `pc=0x800004ec` 触发 `HIT GOOD TRAP`，仿真器返回 0。

每轮完成 256 次点积，总周期如下：

轮次	Baseline 总周期	vdot 总周期
1	5617	2215
2	5555	2170
3	5569	2159
中位数	5569	2170

```
程序加速比 = 5569 / 2170 = 2.566 倍
周期减少比例 = (5569 − 2170) / 5569 = 61.03%
```

Baseline 为 5555 ~ 5617 周期，vdot 为 2159 ~ 2215 周期。

原始数据见 运行日志、周期样本 和 统计结果，程序及仿真器版本见 版本校验记录。

6.2 结果分析

vdot 将符号扩展、乘法和归约合并，块内计算由 4 条指令、10 个微操作减少为 1 条指令、1 个微操作，减少了指令译码、发射及中间结果写回。加载、保存和标量循环的开销保持不变，批量程序总周期减少 61.03%。

在 VLEN=128、LMUL=1、vl=16、6 KiB 数据预热的条件下，vdot 相比 RVV 组合指令获得 2.566 倍程序加速。LMUL=2、4、8 和大数据集测试见第 7 节。完整 LLM 推理性能及新增归约逻辑对工作频率的影响，需进一步测试。

7. LMUL 与大数据集测试

7.1 测试方法

使用 vdot-scale-bench 比较不同 LMUL 和数据量下的批量程序周期。各项均设置 `SEW=8`，使用对应的最大 `vl`：

测试	LMUL	vl	输入块数	每轮点积次数	每轮 INT8 乘法次数	输入与输出大小
m2	2	32	64	128	4096	5 KiB
m4	4	64	64	128	8192	9 KiB
m8	8	128	64	128	16384	17 KiB
large	8	128	512	1024	131072	136 KiB

大数据集的输入与输出共 136 KiB，为一级数据缓存容量的 4.25 倍。m8 与 large 使用相同指令序列，输入块数从 64 增加到 512。

每块输入按固定种子生成，`a[i]=INT8(base+i)`，`b[i]=value`，其中 base、value 为有符号 INT8。参考值按等差数列求和，并扣除 INT8 回绕的影响：

```
int wrapped = base + ELEMENTS - 128;
if (wrapped < 0) wrapped = 0;
int sum = ELEMENTS * base + ELEMENTS * (ELEMENTS - 1) / 2 - 256 * wrapped;
int32_t expected = sum * value;
```

本组 Baseline 采用 `vwmul.vv` 和 `vwredsum.vs`，先得到完整 INT16 乘积，再归约为 INT32。第 4 节的基线使用符号扩展和普通乘法，两组加速比分别按各自基线计算。LMUL=2、4 时，乘积分别占 m4、m8 寄存器组。以 LMUL=4 为例，每块计算序列为：

```
vsetvli zero, t6, e8, m4, tu, mu    # t6=64
vwmul.vv v24, v8, v16
vsetvli zero, t6, e16, m8, tu, mu
vwredsum.vs v4, v24, v0             # v0 在循环前清零
```

INT8 加宽为 INT16 时，乘积的 `EMUL=2×LMUL`。LMUL=8 对应 EMUL=16，超过 RVV 规定的最大寄存器组大小 8，因此将两个源组分别拆为前、后两个 m4 组。先计算 `v8/v16` 的前 64 对元素，再计算 `v12/v20` 的后 64 对元素；第二次归约以第一次的 v4 为初值。两次乘法均使用 v24～v31 保存乘积。目标高 96 位在循环前清零，后续通过 `tu` 保持。

vdot 使用一条 `vdot.vv v4, v8, v16` 处理整个源组。两种实现采用相同的加载、保存和标量循环；向量配置、归约初值、全部计算和访存均计入总周期。

两种实现各预热一遍，再交替测量三轮，每轮遍历全部输入两遍。每次运行结束后检查完整输出，m2、m4、m8 各检查 512 个向量结果，large 检查 4096 个。所有测试均开启 NEMU 差分。

7.2 运行命令

```
cd /home/xpg/cie/xs-env
bash nexus-am/apps/vdot-scale-bench/run.sh
```

单独运行大数据集测试：

```
bash nexus-am/apps/vdot-scale-bench/run.sh large
```

7.3 测试结果

LMUL=2、4、8 的三项测试均通过 512 个完整向量结果检查，NEMU 与 RTL 差分未报错，仿真器均返回 0。周期数据如下：

测试	第 1 轮 Baseline / vdot	第 2 轮 Baseline / vdot	第 3 轮 Baseline / vdot
m2	3055 / 2109	3043 / 2115	3036 / 2149
m4	5461 / 3886	5434 / 3919	5470 / 3934
m8	10439 / 7308	10405 / 7293	10405 / 7305

测试	Baseline 中位数	vdot 中位数	程序加速比	周期减少
m2	3043	2115	1.439 倍	30.50%
m4	5461	3919	1.393 倍	28.24%
m8	10405	7305	1.424 倍	29.79%

测试	运行日志	周期样本	统计结果	版本记录
m2	日志	CSV	JSON	校验值
m4	日志	CSV	JSON	校验值
m8	日志	CSV	JSON	校验值

7.4 结果分析

LMUL=2、4、8 时，vdot 分别拆成 2、4、8 个微操作，通过临时寄存器依次传递部分和。源组增大后，参与计算的元素和微操作数量同时增加，部分和之间存在顺序依赖。三项测试的程序加速比为 1.393 ~ 1.439 倍。首微操作还读取旧目标，供零长度执行时保留原值；本测试连续写入 v4，各块之间也存在旧目标依赖。